CASPO: Learned Prefix Value Modeling for Stepwise Credit Assignment in Reasoning RL

Anonymous

Abstract

We propose CASPO (Credit Assignment Step Policy Optimization), a method for reinforcement learning of mathematical reasoning that obtains step-level credit signals from a learned prefix-value model rather than from on-policy Monte Carlo prefix rollouts. CASPO retains the VinePPO step-segmentation and step-TD advantage structure but replaces VinePPO's $K{=}9$ rollouts at each non-terminal prefix with a single forward pass through a learned process reward model (PRM), trained to predict $P(\text{success} \mid \text{prefix})$ from Monte Carlo step labels. A periodic refresh of the PRM on rollouts of the current policy controls distribution drift across RL training. The result is a strict drop-in for VinePPO that uses the same PPO clipped objective, the same KL penalty, the same response segmentation, and the same global PPO minibatch, but eliminates the per-step Monte Carlo expansion that dominates VinePPO's wall-clock cost. We compare four methods (GRPO, PPO+Critic, VinePPO, CASPO) under one trainer, one verifier, one rollout engine, and one set of hyperparameters, on Qwen2.5-Math-1.5B with the `dsr_sub` subset of DeepScaleR (1209 prompts from One-Shot-RLVR), with DeepSeekMath-7B on MATH-lighteval as a 7B paper-faithful reference. PPO+Critic is the canonical Schulman-2017 PPO with a learned scalar value head on a stripped-LM-head backbone copy and GAE; we use it rather than bare PPO (uniform sequence-broadcast advantages with no value baseline) because GRPO already covers the value-free case via its per-prompt group-relative baseline. PPO+Critic provides a per-token learned baseline that contrasts with CASPO's per-step learned baseline and with VinePPO's on-policy Monte-Carlo step values. Throughout we keep the algorithmic contract identical except for the source of the per-step value $V(s_t)$, isolating the credit-signal change from confounders. Empirical results are reported in Section 6 after the corresponding training and evaluation runs complete.

1 Introduction

Reinforcement learning has emerged as the dominant approach for improving the mathematical reasoning of large language models, but the dominant signal is sparse: a verifier returns 0/1 only at the end of a long chain of thought, leaving the optimizer to assign credit across hundreds of intermediate tokens. PPO [5] addresses this by broadcasting a sequence-level advantage uniformly across response tokens; GRPO [6] refines the variance term using a group-relative baseline. Neither method distinguishes a correct intermediate inference from a lucky guess that nonetheless reached the right final answer.

VinePPO [2] addresses this gap by introducing *step-level* value estimates: the response is segmented into reasoning steps, and the value at each step boundary s_t is estimated by sampling K on-policy continuations from s_t and averaging their terminal verifier rewards. This produces well-calibrated step values that bear out empirically—VinePPO reports substantial improvements over PPO and GRPO on MATH and GSM8K—but at a cost: each non-terminal step boundary requires K additional generations, multiplying wall-clock time by roughly the average step count per response.

This cost is most punishing precisely on the long-chain regimes where step-level credit is most needed. On the One-Shot-RLVR `dsr_sub` DeepScaleR subset with Qwen2.5-Math-1.5B, the LaTeX-aware step splitter produces ∼40 step boundaries per response (vs. ∼10–20 on upstream MATH), and the K=9 Monte Carlo continuations push per-step training to ∼33 minutes on 4×H100 80GB. A 600-outer-step VinePPO run at this configuration thus requires roughly 14 days of wall-clock and is practically infeasible to run as a baseline at our experimental scale; the cost-effectiveness gap to VinePPO is largest exactly where step-level credit assignment matters most.

CASPO replaces those Monte Carlo prefix rollouts with a single forward pass through a *learned* prefix value model $V_\phi(s_t)$, a process reward model (PRM) trained to predict $P(\text{success} \mid s_t)$ from Monte Carlo step labels collected on the SFT policy. CASPO keeps every other VinePPO component fixed: the same LaTeX-aware step splitter, the same step-TD advantage construction, the same clipped PPO objective, and the same KL penalty. Step-level credit is therefore extracted from V_ϕ in $\mathcal{O}(1)$ forward passes per response rather than $\mathcal{O}(K \cdot \text{steps})$ generations, and a periodic *refresh* of V_ϕ on rollouts of the current policy controls the distribution shift induced by RL.

Contributions. We contribute: (i) CASPO: a method that drops VinePPO's prefix Monte Carlo step in favor of a learned process reward model (PRM) trained to predict $P(\text{success} \mid s_t)$ from Monte Carlo step labels, plugged into the VinePPO step-TD advantage construction; (ii) two step-TD advantage formulations of the prefix value: Δp (default), the canonical match for a binary verifier, and $\Delta \log p$ (ablation), which operates on the log-probability scale of the underlying token logits and amplifies rescue signal on the failure tail; (iii) an iterated-refresh configuration that periodically retrains V_ϕ from scratch on Monte-Carlo-labeled rollouts of the current policy, converting the credit model from a single drifting model into a sequence of in-distribution snapshots at amortized wall-clock cost well below one VinePPO outer-step budget; we ablate refresh against the no-refresh default that holds the Phase-1 V_ϕ fixed throughout RL; (iv) a unified comparison stack that implements GRPO, PPO+Critic, VinePPO, and CASPO under one trainer, one verifier, and one rollout engine, controlling everything except the algorithmic credit-signal choice; and (v) implementation infrastructure for full-finetune 7B-scale RL with FSDP-sharded trainer and rank-local vLLM generation, including a streaming CUDA-IPC weight-sync path that is collective with `summon_full_params`, fp32 master weights with on-demand AdamW CPU-offload during the sync window to fit the unsharded fp32 materialization, and per-method ckpt sharding across multiple NVMe drives so each of the four 1B methods runs to completion in parallel without disk contention.

Implementation provenance. The codebase is an adaptation and extension of the public VinePPO setup [2, 4]. The DeepSeekMath-7B configuration's prompt template, rollout shape, PPO hyperparameters, evaluation protocol, and LaTeX-aware step segmentation are matched to or derived from the upstream paper and reference repository so that any wall-clock or accuracy gap between methods reflects the algorithmic change rather than infrastructure. The Qwen2.5-Math-1.5B configuration on `dsr_sub` adapts the same training contract to the model's native prompt template and to a 4-GPU FSDP topology, and is the primary experimental site for the comparison and for the iterated-refresh contribution.

2 Related Work

Policy gradient with sparse rewards. PPO [5] optimizes a clipped policy-gradient surrogate to bound the per-step policy update. Applied to language models with verifier-only terminal rewards, the standard practice is to broadcast the sequence-level advantage uniformly across response tokens and add a KL-to-reference penalty to prevent the policy from drifting far from the SFT initialization.

Group-relative baselines. GRPO [6] samples G responses per prompt and uses the within-group reward statistics as the advantage baseline. This avoids training a separate value network entirely, at the cost of treating every intermediate token in a correct response as equally important.

Step-level credit assignment via Monte Carlo prefix values. VinePPO [2] introduces stepwise value estimates by sampling K on-policy continuations from each non-terminal prefix and averaging the verifier outcomes of those continuations. Step advantages are then formed as temporal-difference differences between consecutive prefix values. The reasoning-credit gain is well demonstrated on MATH, but the method's rollout multiplier scales as K times the average number of steps per response.

Learned process supervision. A line of work trains a separate process reward model (PRM) on human-annotated or self-supervised step-correctness labels. CASPO's PRM is in this family: it predicts $P(\text{success} \mid \text{prefix})$ from automatically-generated Monte Carlo step labels (a held-out batch of J rollouts is averaged to form the soft target at each prefix), avoiding the human-annotation bottleneck of PRM800K-style data and recovering a directly calibrated correctness probability suitable for TD-style step advantages.

Implicit prefix value learning (IPVRM). IPVRM [1] formalizes prefix values as cumulative log-ratios of a trainable prefix policy versus a frozen reference policy, trained from trajectory-level outcome labels using a margin BCE. We *adopt IPVRM's cumulative-log-ratio architecture* for V_ϕ and its framing as a calibrated correctness predictor that drives a step-TD advantage. Our differences are in *training-data form* and *refresh strategy*: we train V_ϕ on dense Monte Carlo step labels with a continuous-target BCE (rather than trajectory-level labels with the margin term), and we replace inline online updates with periodic full retraining of V_ϕ from scratch on current-policy rollouts. The choice of parameterization is regime-dependent rather than uniformly favorable: on a leaky prefix-shuffle validation set, the log-ratio form scores val $\rho \approx 0.78$ versus $\rho \approx 0.43$ for a single sigmoid head $V_\phi(s_t) = \sigma(W h_\phi(s_t))$ at matched hyperparameters, but this gap narrows on prompt-level held-out evaluation and inverts on a fully out-of-distribution probe (DeepScaleR prompts disjoint from the `dsr_sub` training set), where the sigmoid head leads log-ratio by $\approx 0.13\ \rho$. On the in-distribution policy-drifted (metric-B) probe at step_300 and beyond, sigmoid-head PRMs cross from small-positive to small-negative ρ (-0.07 to -0.24 at step_300+), i.e. they *anti-correlate* with success at the policy states that drive late-RL credit assignment, while the log-ratio form holds $\rho > 0$ at all measured stages. The log-ratio's $V_\phi(s_t) = \beta \sum_{i<t} \log\big(\pi_\phi/\pi_{\text{ref}}\big)(a_i \mid s_{<i})$ decomposition yields one direct gradient term per response token rather than a single end-of-prefix gradient, which favors fitting the in-distribution policy-drifted prefixes the PRM is asked to score during RL; we discuss this regime-dependence and its implications for deployment in Section 8.

Per-token learned-critic baselines. The classical PPO-with-critic recipe [5] uses a learned scalar value head $V_\psi(s,t)$ trained jointly with the policy on the value loss $(V_\psi - \hat{R})^2$ and folded into the advantage via generalized advantage estimation (GAE). We include this recipe as a baseline (PPO+Critic) so that CASPO's learned *step-level* prefix value can be compared not only against the value-free baselines (PPO, GRPO) and the on-policy Monte Carlo step value (VinePPO), but also against an in-line per-token learned baseline trained from the same trajectories under the same PPO objective.

Generation backends and weight sync. We use vLLM [3] for high-throughput rollout and prefix caching. The implementation uses CUDA-IPC weight transfer between trainer ranks and rank-local vLLM engines; for the FSDP-sharded 7B trainer, the IPC sync is implemented as a collective `summon_full_params` block over the FSDP-wrapped policy followed by streaming per-tensor IPC handle exchange with each rank's vLLM `EngineCore`, in chunks of 8 parameters with per-parameter bf16 casting to bound transient host memory during the sync window.

3 Background

3.1 Reasoning RL setup

A trajectory consists of a problem prompt x and a model response $y = (y_1, \ldots, y_T)$ generated autoregressively. A verifier $R(x, y) \in \{0, 1\}$ returns 1 if and only if the boxed final answer in y matches the ground truth. The training objective is to maximize $\mathbb{E}_{x\sim\mathcal{D},y\sim\pi_\theta(\cdot|x)}R(x, y)$ subject to a KL constraint to a frozen reference policy π_{ref}, which we instantiate as the SFT initialization.

3.2 Step segmentation

We split each response into reasoning steps using the LaTeX-aware splitter of VinePPO [2, 4], which identifies natural mathematical step boundaries (display equations, aligned blocks, sentence terminators in prose). Let $0 = t_0 < t_1 < \cdots < t_S = T$ be the resulting boundary token indices and let $s_t \triangleq y_{<t}$ denote the prefix state at boundary t.

3.3 Step-TD advantage

Given a per-step value function $V(s_t)$, both VinePPO and CASPO construct the step-TD advantage

$$A(s_t) = r_t + \gamma\, V(s_{t+1}) - V(s_t), \qquad r_t = \begin{cases} R(x,y) & t = S, \\ 0 & t < S, \end{cases} \tag{1}$$

with $\gamma = 1$. CASPO obtains V from a learned model V_ϕ; VinePPO obtains V by averaging K Monte Carlo continuations. The downstream PPO objective and KL penalty are identical for both methods; only the source of V differs.

3.4 PPO objective

We use the clipped PPO objective

$$\mathcal{L}_{\text{clip}}(\theta) = -\,\mathbb{E}_t\big[\min\big(\rho_t(\theta)\, A_t,\ \text{clip}\big(\rho_t(\theta), 1-\epsilon, 1+\epsilon\big)\, A_t\big)\big]\,, \tag{2}$$

with $\rho_t(\theta) = \pi_\theta(y_t \mid x, y_{<t})/\pi_{\theta_{\text{old}}}(y_t \mid x, y_{<t})$ and $\epsilon = 0.2$. A KL-to-reference term is added with the k_3 estimator:

$$\text{KL}_{k_3}(\pi_\theta \,\|\, \pi_{\text{ref}}) \approx \mathbb{E}_t[\exp(\delta_t) - \delta_t - 1]\,, \quad \delta_t = \log \pi_{\text{ref}}(y_t) - \log \pi_\theta(y_t). \tag{3}$$

The advantages A_t in (2) are (a) terminal-broadcast group-relative advantages for GRPO; (b) GAE per-token advantages from a learned scalar value head for PPO+Critic; or (c) the step-TD advantages of (1) broadcast over the tokens in each step span for VinePPO/CASPO.

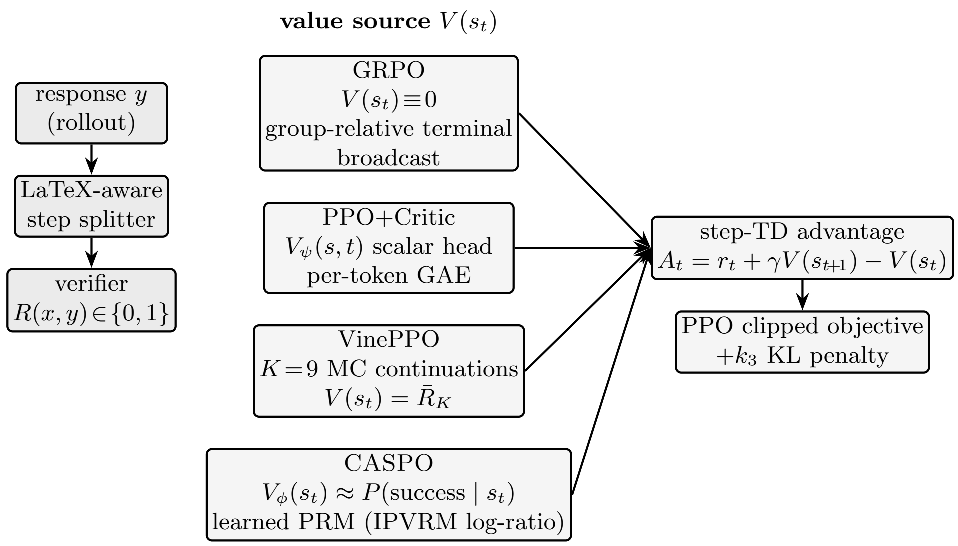

Figure 1: Four compared methods, anchored on the source of the per-step value $V(s_t)$. Rollout, segmenter, verifier, advantage form, PPO clipped objective, and KL penalty are identical across all rows; the only difference is which black box produces $V(s_t)$ used in $A_t = r_t + \gamma V(s_{t+1}) - V(s_t)$. GRPO uses no value ($V \equiv 0$) with a per-prompt group-relative terminal broadcast. PPO+Critic (the canonical Schulman-2017 PPO) learns a scalar per-token value head trained on $(V_\psi - \hat{R})^2$ with GAE. VinePPO estimates $V(s_t)$ by averaging K=9 on-policy continuations from each prefix. CASPO replaces those continuations with a learned process reward model $V_\phi(s_t)$ trained on Monte Carlo step labels to approximate $P(\text{success} \mid s_t)$, with periodic refresh on current-policy rollouts (§4.1, §4.4).

4 Method: CASPO

Figure 1 summarizes the comparison axis: rollout, step splitter, verifier, advantage form, PPO objective, and KL penalty are held identical across all four methods. Only the source of the per-step value $V(s_t)$ differs.

4.1 Prefix value model

The prefix value model V_ϕ is a neural network trained to predict the probability that a rollout starting from prefix s_t eventually reaches a correct final answer:

$$V_\phi(s_t) \;\approx\; P\big(R(x,y)=1 \mid s_t\big). \tag{4}$$

Following IPVRM [1] we parameterize V_ϕ as a *cumulative log-ratio* of a trainable prefix policy π_ϕ against a frozen reference policy π_{ref} (both initialized from the same SFT checkpoint as the base policy):

$$V_\phi(s_t) \;=\; \beta \sum_{i<t} \log \frac{\pi_\phi(a_i \mid s_{<i})}{\pi_{\text{ref}}(a_i \mid s_{<i})}, \tag{5}$$

with $\beta = 10$ a fixed stability scale. The PRM probability is recovered by a sigmoid on the scaled logit $\sigma\big(V_\phi(s_t)/\beta\big) \in [0,1]$. At initialization $\pi_\phi = \pi_{\text{ref}}$, so $V_\phi \equiv 0$ everywhere and the implied probability is exactly 0.5 for every prefix — a natural calibrated prior. We refer to V_ϕ as the *process reward model* (PRM), following the convention of Gao et al. [1] and the process-supervision literature. The cumulative-log-ratio form is critical *at the deployment regime* (in-distribution policy-drifted prefixes the PRM is asked to score during RL): because the value at any prefix decomposes as a sum of per-token log-ratio terms, every response token contributes a direct gradient signal during training rather than only the single-token signal an end-of-prefix scalar head would provide. We verified empirically that on the leaky prefix-shuffle validation set, replacing this parameterization with $V_\phi(s_t) = \sigma(W h_\phi(s_t))$ at matched hyperparameters collapses validation Spearman ρ from ≈ 0.78 to ≈ 0.43. On the metric-B held-out evaluation (in-distribution policy-drifted prefixes), log-ratio leads sigmoid by $\approx 0.10-0.13$ ρ across all CASPO checkpoints we measure; on a fully out-of-distribution probe the relationship inverts (§8).

4.2 Phase-1: PRM training via Monte Carlo step labels

We train V_ϕ directly on Monte Carlo step labels rather than via a trajectory-level BCE-with-margin objective. For each of N training prompts we sample K base rollouts from the SFT policy at temperature 1.0. From each rollout we randomly select S step boundaries; at each boundary s_t we sample J additional independent continuations from the same SFT policy, each scored by the verifier $R \in \{0, 1\}$, and form the empirical Monte Carlo estimate

$$\widehat{p}(s_t) \;=\; \frac{1}{J} \sum_{j=1}^{J} R\big(x, [s_t; y^{(j)}_{>t}]\big), \qquad y^{(j)}_{>t} \sim \pi_{\mathrm{SFT}}(\cdot \mid s_t), \tag{6}$$

which is an unbiased estimator of $P(R = 1 \mid s_t)$. Following the process-supervision practice of mixed-outcome filtering, we discard rollouts whose J continuations are all 0 or all 1 at every sampled boundary, since saturated prefixes contribute no learning signal.

We train V_ϕ to minimize binary cross-entropy against the continuous targets $\widehat{p}(s_t) \in [0, 1]$, applied to the sigmoid of the scaled cumulative log-ratio:

$$\mathcal{L}_{\mathrm{PRM}}(\phi) \;=\; -\mathbb{E}_{s_t}\Big[\widehat{p}(s_t)\, \log \sigma\big(V_\phi(s_t)/\beta\big) \;+\; \big(1 - \widehat{p}(s_t)\big)\, \log\big(1 - \sigma\big(V_\phi(s_t)/\beta\big)\big)\Big], \tag{7}$$

with $V_\phi(s_t)$ defined by the cumulative log-ratio of (5). Equation (7) is BCE on continuous targets in $[0, 1]$ rather than binary; it minimizes the Bregman divergence between the implied PRM probability $\sigma\big(V_\phi(s_t)/\beta\big)$ and the MC label $\widehat{p}(s_t)$, and recovers the same calibration target without needing the trajectory-level margin term of IPVRM [1]. Concretely, the trainer caches the per-token log-probabilities of π_ϕ and π_{ref} on the prefix, forms the per-token log-ratios, takes a cumulative sum, and gathers the value at the step boundary s_t. Because all $i \le t$ log-ratio terms enter the loss additively, every response token in the prefix produces a direct gradient signal — which is the empirical reason this parameterization dominates an end-of-prefix scalar-head alternative on the leaky validation set and on the in-distribution policy-drifted regime that drives RL (§2, §8). We train for three epochs with full-model FSDP updates, $\beta = 10$, effective batch size 32, and learning rate 5×10^{-6}. Best-by-validation checkpoint selection operates on a leaky prefix-shuffle val for runtime efficiency; we cross-validate the selected checkpoint on a non-leaky prompt-level held-out probe at the end of training (§4.2).

Held-out PRM evaluation. For PRM quality reporting we evaluate V_ϕ on a *prompt-level* held-out split rather than a prefix-level shuffle. The default `train_value_mc.py` flow exposes two non-leaky modes: `--split_by_prompt`, which holds out $\lceil \text{val\_fraction} \cdot |\mathcal{D}_{\text{prompt}}| \rceil$ unique prompts (and all their prefixes) from the training collection, and `--held_out_data <path>`, which loads a separately-collected mc_labels file generated on disjoint prompts (e.g. a fresh DeepScaleR slice disjoint from `dsr_sub`). The prefix-level row shuffle is preserved as a legacy default but flagged explicitly as leaky in the trainer's startup banner. The Phase-1 PRM that drives our RL runs (the "orig" recipe) was collected over the full `dsr_sub` (N=1209 prompts, K=16, J=16, sharded across four parallel collection processes), of which 1208 unique prompts were observed across shards and 750 unique prompts produced mixed-outcome MC labels (≈62% yield at the SFT base policy). A subsequent policy-source refresh collected on rollouts of the step_150 RL checkpoint of `dsr_sub` sees the same 1208 unique prompts with 561 mixed-outcome (≈46% yield), reflecting the policy moving away from chance success on a subset of prompts as RL proceeds.

4.3 Phase-2: step-TD policy optimization

At each PPO outer step, the trainer samples G responses per prompt for P prompts, segments each response, and constructs step values $\{V_\phi(s_t)\}$ in a single forward pass through V_ϕ using (4). Step advantages are assembled from (1) and broadcast over the tokens within each step span before applying the PPO objective in (2) with the KL penalty in (3). The default CASPO run holds V_ϕ frozen during policy optimization; an iterated-refresh variant (next section) periodically retrains V_ϕ from scratch on rollouts of the current policy to control distribution drift.

4.4 Iterated PRM refresh

PRM decay (and recovery). A learned PRM is in-distribution only at the policy state on which its MC labels were collected. As RL training updates the policy, the prefixes seen at outer step t drift away from the policy that generated the Phase-1 labels, and V_ϕ's calibration on those prefixes can deteriorate. We measure this directly on Qwen2.5-Math-1.5B with the `dsr_sub` dataset by computing the per-checkpoint Spearman ρ between $V_\phi(s_t)$ and a fresh K=4, J=8 MC estimate of $P(\text{success} \mid s_t)$ at policy states {step_50, step_100, ..., step_400}, generated at the same response cap as the PRM's training data (cap=1024).

The Phase-1 PRM trained on SFT-policy rollouts behaves *non-monotonically* across RL stages: ρ rises through early RL, peaks at step_150 (ρ=0.54), drops sharply at step_200–250 (ρ=0.23, 0.20) as the policy executes a structural change in its reasoning patterns, and partially recovers thereafter (ρ=0.27, 0.32, 0.38 at step_300/350/400). The drop at step_200–250 is the largest single calibration loss and is what motivates a refresh trigger before that window. The partial recovery from step_300 onward suggests that the policy's distribution re-stabilizes after the structural transition, but V_ϕ never re-attains its step_150 peak. PRM staleness is therefore better described as "staircase decay with a critical mid-RL drop" than as smooth monotone drift, and the refresh schedule should be tuned to catch the drop window rather than to a fixed pace.

Refresh mechanism. We address PRM decay by replacing a single drifting V_ϕ with a sequence of in-distribution snapshots, refreshed periodically. Every k outer steps the trainer pauses, collects a new MC label set from the current policy snapshot under the same Phase-1 recipe ((6), (7)), retrains V_ϕ *from scratch* on those labels (initialized from the SFT backbone, not from the previous V_ϕ, to avoid bias compounding across refreshes), and resumes RL using the new V_ϕ as the prefix value source. The policy resumes with its AdamW first- and second-moment buffers and its learning-rate scheduler state preserved across the refresh boundary, so optimizer warm-up and lr decay continue along the original 600-step horizon rather than restarting at each refresh. We use $k = 150$ for the Δp formulation and $k = 200$ for $\Delta \log p$, chosen so that ρ at the refresh trigger remains well above the empirical $\tau \approx 0.4$ floor at which the credit-signal contribution of V_ϕ becomes negligible.

Implicit regularization from policy-source labels. A side benefit of the refresh design is that it makes the (otherwise leaky) prefix-shuffle val align with the metric-B (policy-drifted prefix) probe used to validate V_ϕ for RL. The Phase-1 PRM trained on SFT-base rollouts *overfits* the leaky val past $\sim$3000 optimizer steps: leaky val keeps decreasing while metric-B ρ on policy-drifted prefixes decays. A refresh PRM trained on rollouts of the step_150 RL checkpoint does *not* exhibit this asymmetry — its metric-B ρ trajectory is roughly flat from step $\sim$500 to the end of training, mirroring its leaky-val trajectory. The mechanistic explanation is direct: leaky val measures fit to the training distribution, and for the Phase-1 PRM the training distribution (SFT-base policy) is not the deployment distribution (RL-current policy), so leaky val mis-selects checkpoints; for the refresh PRM the training distribution $\approx$ deployment distribution and leaky val $\approx$ metric-B. This validates the refresh design as having implicit regularization beyond the obvious policy-distribution match: the same trainer recipe that we previously diagnosed as leaky becomes a usable selector once the source policy is moved on-policy.

Per-checkpoint PRM specialization. Comparing the Phase-1 PRM and the policy-source refresh PRM checkpoint-by-checkpoint along the RL training trajectory shows that the two are not uniformly ranked: the Phase-1 (SFT-source) PRM wins early-RL prefixes (step_0 through step_150) by $\approx +\ 0.12\ \rho$ on metric-B, while the refresh PRM wins mid-to-late-RL prefixes (step_150+) by $\approx +\ 0.08\ \rho$. Both gaps are real per-checkpoint signals (more than three to four standard errors on the per-stage estimate) but cancel when averaged over the eight CASPO RL checkpoints we log, so the trajectory-averaged comparison is essentially a tie. We therefore report a softer claim than "refresh dominates": the refresh PRM is the right choice for RL credit signals after the policy has moved appreciably from the SFT base, and the Phase-1 PRM remains the right choice in the early-RL window before that drift has occurred.

Refresh cost. A single refresh on Qwen2.5-Math-1.5B with `dsr_sub` (FSDP=4, N=1209 prompts, three epochs, effective batch 32) takes $\approx$ 57 minutes on 4×H100 80GB at the policy-source recipe ($\approx$ 26 min for MC collection at K=16, J=8, cap=1536, plus $\approx$ 31 min for the PRM training pass). The Phase-1 PRM trained under the original recipe (K=16, J=16, cap=1024) runs longer at $\approx$ 88 minutes ($\approx$ 48 min collection plus $\approx$ 40 min training). At k=150 the refresh amortizes to $\approx$17.7 s of refresh overhead per RL outer step (three mid-RL refreshes over a 600-step run); at k=200 it amortizes to $\approx$11.8 s/step (two refreshes). The amortized refresh cost is small relative to the $\sim$78 s/step CASPO RL step time (Section 5.5) and is an order of magnitude below VinePPO K=9.

Ablation. We report CASPO both with and without refresh: CASPO (the no-refresh default holds the Phase-1 V_ϕ fixed throughout RL) and CASPO+refresh (the schedule above). The two variants share every other component (PPO objective, KL, advantage transform, segmenter), isolating the refresh contribution to a single ablation.

4.5 Step-TD advantage formulations from V_ϕ

A modeling choice in CASPO is *which* statistic of V_ϕ enters the step-TD difference. Because $V_\phi(s_t) \in [0, 1]$ is already a calibrated correctness probability under (4), two natural formulations arise: the TD on probabilities (Δp, default) and the TD on log-probabilities ($\Delta \log p$, ablation).

Δp (default, probability-scale TD):

$$A^{\Delta p}(s_t) \ = \ r_t \ + \ \gamma\, V_\phi(s_{t+1}) \ - \ V_\phi(s_t). \tag{8}$$

$\Delta \log p$ (ablation, log-probability-scale TD):

$$A^{\Delta \log p}(s_t) \;=\; r_t \;+\; \gamma \, \log V_\phi(s_{t+1}) \;-\; \log V_\phi(s_t). \tag{9}$$

In both variants the terminal verifier reward $r_T = R(x, y)$ enters identically; the only difference is whether the TD operates on V_ϕ or $\log V_\phi$.

Intuition.

- Δp: The TD operates on the probability scale, so $|A^{\Delta p}(s_t)| \leq 1$ before standardization. Because $V_\phi(s_t) \approx P(R = 1 \mid s_t)$, $A^{\Delta p}$ is the empirical advantage an idealized binary-reward critic would compute: the change in expected terminal reward attributed to step t. This matches the binary verifier exactly. Its main weakness is *near-saturation*: when $V_\phi(s_t)$ is close to 0 or 1, Δp contributions are vanishingly small, so highly confident prefixes contribute little gradient.

- $\Delta \log p$: The TD operates on the log-probability scale, mirroring the form of token log-likelihoods used in the PPO ratio $\rho_t(\theta)$. For a sequence model trained by maximum likelihood, log-probability differences are the canonical update signal; using $\log V_\phi$ keeps the value-derived advantage on the same information-theoretic scale as the underlying token logits. The log-transform also amplifies the *rescue* signal: a step that moves a near-failed trajectory ($V_\phi \to 0$) toward feasibility contributes a large negative-side gradient under $\log V_\phi$ but only a small positive-side gradient under V_ϕ. The trade-off is unbounded magnitude on the failure tail, which we control with the shared ± 3 standardized advantage clip in Section 4.

When each formulation is preferable. Δp is the default: it is the canonical match for a binary verifier and has the clean expected-terminal-reward interpretation $A^{\Delta p}(s_t) = r_t + \gamma \, \mathbb{E}[R \mid s_{t+1}] - \mathbb{E}[R \mid s_t]$ via (4). $\Delta \log p$ is an ablation that exchanges saturation-near-1 robustness for unbounded-magnitude gradients on the failure tail; we report it as the second formulation. We report both on MATH-500 in Section 6.

4.6 Step advantage normalization and clipping

Let $\mathcal{A} = \{A(s_t)\}$ over all valid step boundaries in the outer PPO step. We standardize over the entire batch ($\widehat{A}_t = (A_t - \mu_\mathcal{A})/(\sigma_\mathcal{A} + \varepsilon)$) and clip the standardized advantages to $[-3, 3]$ to bound the contribution of single-step outliers. The clip range is chosen empirically to balance suppression of pathological prefixes against retaining genuine step-level signal.

5 Experimental Setup

5.1 Models and data

The primary experimental site is **Qwen2.5-Math-1.5B** (`Qwen/Qwen2.5-Math-1.5B`), a 1.5B math-specialist SFT initialization, trained on the **One-Shot-RLVR** `dsr_sub` 1209-prompt subset of DeepScaleR. Evaluation uses MATH-500, GSM8K, and OlympiadBench under greedy decoding (k=1, T=0). Qwen2.5-Math is a math-specialist SFT model that responds best to its native template `{query}\nLet's think step by step and output the final answer within \boxed{}.`; substituting the `[MATH_TASK]` template degrades Qwen accuracy by >30 percentage points and is a critical reproduction footgun.

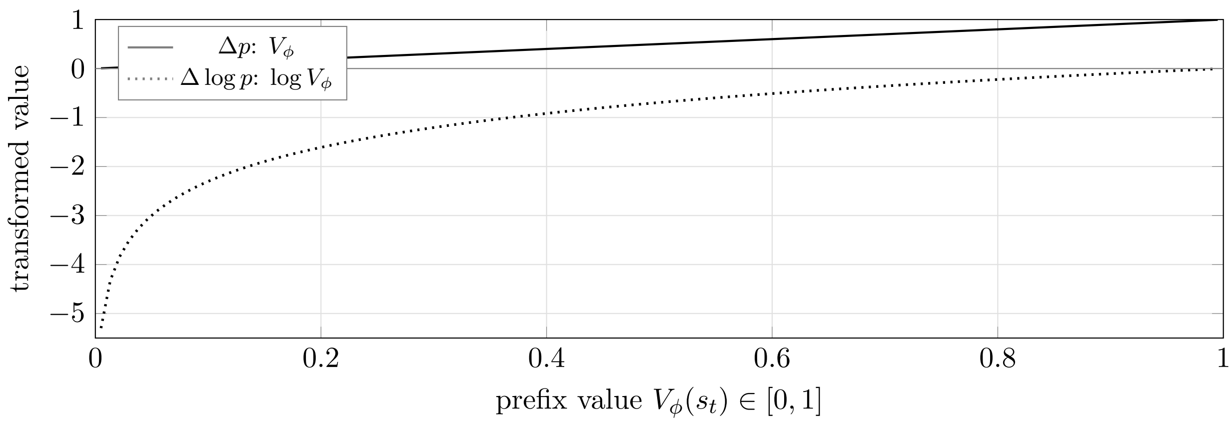

Figure 2: The two step-TD transforms of the prefix value $V_\phi(s_t) \in [0, 1]$. Δp (solid) is the identity: bounded magnitude $|A^{\Delta p}| \leq 1$ before standardization, but saturates near $V_\phi = 0$ and $V_\phi = 1$ where two prefixes the model is confident about contribute essentially the same gradient. $\Delta \log p$ (dotted) is unbounded as $V_\phi \to 0$, amplifying *rescue* signal on the failure tail (where most early-training gradient lives) and compressing the high-confidence regime. Both transforms share the global advantage clip $|A| \leq 3$ in Section 4. The TD difference $A^*(s_t) = r_t + \gamma\, f(V_\phi(s_{t+1})) - f(V_\phi(s_t))$ inherits the regime of f.

For a 7B paper-faithful reference site, we additionally evaluate on **DeepSeekMath-7B** (`realtreetune/deepse` the SFT-on-MATH initialization from the upstream VinePPO pipeline, trained on `DigitalLearningGmbH/MATH-li` (train split). 7B held-out evaluation uses `HuggingFaceH4/MATH-500` (test split, 500 problems) plus the full MATH test set, College Math, and OlympiadBench (math-only configuration), under the upstream prompt template `[MATH_TASK] Problem:\n{query}\n\nSolution:` and the LaTeX-aware step splitter from the upstream codebase.

5.2 Methods

We compare four methods (Figure 1):

GRPO Per-prompt group-relative terminal-reward advantages with $G = 8$. Stands in for the value-free baseline; bare PPO (uniform sequence-broadcast advantages, no group baseline, no value head) is strictly weaker on this evaluation regime and is omitted.

PPO+Critic Schulman-2017 PPO with a learned scalar value head V_ψ on a stripped-LM-head backbone copy, trained jointly with the policy under a clipped value loss $\max\big[(V_\psi - \hat{R})^2,\ (\mathrm{clip}(V_\psi, V_\psi^{\mathrm{old}} \pm \epsilon_v) - \hat{R})^2\big]$ with $\epsilon_v = 0.2$ and $\lambda_{\mathrm{GAE}} = 1.0$, value-loss coefficient 1.0, critic LR 10^{-6}.

VinePPO $K = 9$ Monte Carlo continuations from each non-terminal step boundary; step-TD advantages.

CASPO Learned process reward model $V_\phi(s_t) \approx P(\text{success} \mid s_t)$, an IPVRM-style cumulative-log-ratio network trained on Monte Carlo step labels (Section 4.2), plugged into the step-TD advantage. We report two step-TD formulations (Section 4.5): CASPO-Δp (default) and CASPO-$\Delta \log p$ (ablation). Each is reported with and without the iterated PRM-refresh schedule of Section 4.4, labeled CASPO and CASPO+refresh respectively, where the no-refresh default holds the Phase-1 V_ϕ fixed throughout RL.

Table 1: Training hyperparameters at the two experimental sites. The Qwen2.5-Math-1.5B configuration is the primary site for the comparison; DeepSeekMath-7B retains the upstream VinePPO configuration as a paper-faithful 7B reference. The 2048-token response budget for Qwen2.5-Math-1.5B is justified empirically (see text below).

Field	Qwen2.5-Math-1.5B	DeepSeekMath-7B
Group size G	8	8
Prompts per outer step	128	64
Responses per outer step	1024	512
Global PPO minibatch	128	64
PPO epochs per rollout	2 (GRPO reported at both μ=1 and μ=2[a])	2
Policy LR	1×10^{-6}	1×10^{-6}
Online value LR (CASPO)	1×10^{-6}	1×10^{-6}
Critic LR (PPO+Critic)	1×10^{-6}	1×10^{-6}
Value β, m	10, 5	10, 5
PPO clip ϵ	0.2	0.2
Value clip ϵ_v (PPO+Critic)	0.2	0.2
GAE λ (PPO+Critic)	1.0	1.0
KL coefficient	10^{-3} (k_3, 10^{-2} for PPO+Critic)	10^{-2} (k_3)
Advantage clip	3.0	3.0
Advantage standardization	batch	batch
Rollout sampling	temp 0.6, top-p 1.0	temp 0.6, top-p 0.9
Max prompt / response	1024 / 2048	1512 / 1024
Max sequence length	3072	2499
Warmup steps	0	480
Outer steps	600	1000
Checkpoint cadence	every 50	every 200

[a] PPO+Critic, VinePPO, and CASPO are fixed at μ=2 epochs per rollout, matching the upstream VinePPO configuration. GRPO is reported at both μ=1 (canonical setting in DeepSeekMath [6], TRL, and verl) and μ=2 (matches the optimizer-step budget per rollout of the other three methods, following the VinePPO-paper GRPO baseline). The μ=1 comparison is GRPO-faithful; the μ=2 comparison isolates the credit-signal change from optimizer-budget effects.

5.3 Hyperparameters

All four methods share the same PPO infrastructure. Table 1 summarizes the key knobs.

The Qwen2.5-Math-1.5B configuration uses 1024 responses per outer step (128 prompts $\times G$=8) on a 4-GPU FSDP topology. The KL coefficient of 10^{-3} for CASPO/GRPO/VinePPO (versus 10^{-2} for PPO+Critic) is empirically required: the math-specialist Qwen SFT initialization is over-anchored at 10^{-2} and policy updates do not move the model without a weaker reference anchor, while PPO+Critic is critic-stabilized and tolerates the stronger anchor. The DeepSeekMath-7B configuration retains the upstream VinePPO hyperparameters at 7B [2, 4]: 512 responses per outer step ($64 \times G$=8), 2 PPO epochs per rollout, and a 64-response global PPO minibatch, isolating any empirical differences between the two sites to the model and dataset rather than the optimization recipe.

The response budget of 2048 tokens for Qwen2.5-Math-1.5B is set by empirical measurement on `dsr_sub`: an 800-chain uncapped sample (T=1.0, max=3000) places the 98th percentile of *correct* chains at 1613 tokens. At 2048, only 2% of correct chains are truncated, while the overall truncation rate is 8% (i.e., the truncated tail is overwhelmingly off-track / rambling chains rather than length-bound correct ones). This makes the unfinished-response penalty signal load the right pattern: "long without resolution" is what gets penalized, not "hard problem requires more tokens." Going further to 3072 catches fewer than one additional correct chain per 150 sampled and risks the Qwen `max_position_embeddings`=4096 ceiling at prompt + response.

5.4 Evaluation protocol

Final evaluation reports pass@1 and pass@k ($k = 16$) on MATH-500, full MATH test, College Math, and OlympiadBench. Sampling at evaluation uses temperature 0.35 and top-p 0.9, matching the upstream protocol. Generation is performed with vLLM at `kv_cache_dtype="auto"` (bf16, matching the policy weight dtype) and `gpu_memory_utilization=0.85` for maximum eval throughput on a dedicated GPU. Sample evaluation at intermediate checkpoints uses `EVAL_LIMIT=100`, `EVAL_K=8` for low-overhead training-curve visibility; the full benchmark suite is run at the final checkpoint.

5.5 Infrastructure

Trainer + colocated rollout. Both scales share a single PyTorch trainer plus rank-local vLLM rollout. The 1B configuration uses a single H100 per method (Figure 3, left); the 7B configuration uses FSDP=4 (full_shard, use_orig_params) for GRPO/PPO+Critic/CASPO and FSDP=8 for VinePPO, with vLLM colocated rank-local in both layouts (Figure 3, right).

Streaming CUDA-IPC weight sync. At 1B with a single-rank trainer, the IPC handle exchange is direct. At 7B under FSDP, the trainer enters `FSDP.summon_full_params(writeback=False)` collectively, then streams parameters to each rank's vLLM `EngineCore.collective_rpc("update_weights", ...)` in chunks of 8 tensors with per-parameter bf16 cast inside the IPC push. Streaming bounds peak host-side staging memory and avoids materializing the entire unsharded fp32 policy on host before transfer.

fp32 master weights with on-demand AdamW offload. Both methods co-train policy (and CASPO's value model, or PPO+Critic's critic) in bf16 compute with fp32 master weights. Under FSDP, the master copy lives in the AdamW optimizer state and is materialized into bf16 shards through `MixedPrecision(param_dtype=bf16, reduce_dtype=fp32)`. At sync time the `summon_full_params` block briefly materializes a ∼28 GiB unsharded fp32 buffer per rank for the 7B policy. To fit alongside the already-resident ∼14 GiB of AdamW state and the ∼24 GiB vLLM KV cache reservation per H100, the trainer offloads the AdamW state to pinned host memory before entering the summon block and restores it after IPC push, costing ∼400–500 ms per sync at PCIe Gen5 but freeing the GPU memory required for the unsharded fp32 materialization.

1B fp32 master via autocast. At 1B (single-GPU, no FSDP) the trainer wraps each forward in `torch.autocast(dtype=bfloat16, cache_enabled=False)` so the fp32 parameters cast to bf16 for compute on every op. The non-cached cast is required for AdamW optimizer-state coverage to include all parameters; with caching enabled, the autocast layer holds bf16 tensor wrappers that are not written through to the underlying fp32 parameters during the optimizer step.

Multi-drive checkpoint sharding for parallel 1B training. Each method's 1B run produces ∼15–29 GB per checkpoint (model in bf16, plus the AdamW master+momentum bundles for policy, plus either CASPO's value-model AdamW or PPO+Critic's critic AdamW). At `save_every`=100 over 1000 outer steps that is roughly 150–290 GB per method-run. To run all four 1B method-runs (GRPO, PPO+Critic, CASPO, CASPO-*prob*) concurrently on four GPUs of the same node at the `save_every`=100 cadence, each method's outputs are routed to a dedicated 350 GB NVMe volume, so the four runs neither compete for disk bandwidth nor cross-fill each other's drives.

Reference-policy sharing. The frozen reference policy is shared between the KL term and CASPO's value-model ref to avoid a duplicate forward. All forward passes (rollout, old-logprob rescore, ref-logprob precompute, policy update, value/critic forward) use FlashAttention-3 at 7B on Hopper and FlashAttention-2 at 1B/1.5B.

Microbatch and memory budget on 4-GPU 1.5B FSDP. On the 1.5B FSDP=4 configuration we use a per-rank policy microbatch of 8 with gradient accumulation 4 (preserving the global PPO minibatch of 128). Peak reserved memory is $\sim$17 GiB at PPO+Critic and $\sim$13–15 GiB for the other methods, within the H100 80 GB budget alongside the per-method vLLM KV reservation (`gpu_memory_utilization` 0.45 for CASPO/GRPO, 0.35 for PPO+Critic, 0.50 for VinePPO; `kv_cache_dtype` bf16). With these settings the measured CASPO-Δp outer-step time on Qwen2.5-Math-1.5B (FSDP=4, G=8, 128 prompts/step, mb=8, accum=4, FA2) is $\approx$78 s/step including rollout, old-logprob rescore, value forward, ref-logprob precompute, policy update, and IPC sync.

Cached frozen-reference logprobs for PRM training. PRM training under FSDP=4 with the cumulative-log-ratio parameterization caches the frozen-reference log-probabilities once per prefix at the start of each epoch and replays them from a tensor cache during the gradient pass, avoiding a redundant π_{ref} forward at every microbatch. The cached values are exact bf16 copies of the original forward output, so the cache has no effect on the optimizer trajectory.

Paper-faithful alignment with upstream VinePPO. A targeted audit against the upstream VinePPO reference repository [4] produced a set of small but consequential alignment fixes that we include in the comparison stack so that our VinePPO baseline reproduces the upstream numerics: prepending the BOS token at both rollout and training, per-microbatch advantage whitening, adding the prompt-style stop string `"\n\n\nProblem:"` for vLLM, clamping per-token KL into $[0, 10]$ before reduction, penalizing unfinished sequences via the max-sequence-length policy in (2), an explicit PPO ratio safety skip at 10×, and uniform grad-accumulation per outer step. With these in place, the VinePPO baseline in our stack matches the upstream configuration field-for-field at the respective scale.

Stabilization deviations from upstream. Two deliberate departures from upstream-faithful numerics were required to keep our 1B runs stable. Table 1 reports our 1B KL coefficients: 10^{-3} for GRPO, VinePPO, and CASPO; 10^{-2} for PPO+Critic. The 10^{-3} value is itself an order of magnitude above upstream VinePPO's 10^{-4}, but is the smallest value at which our 1B runs hold KL bounded across 600 outer steps; with upstream's 10^{-4} and no advantage clip, all four 1B methods drift to KL$\to\infty$ within ~ 20 outer steps in our reimplementation. The further 10× bump to 10^{-2} for PPO+Critic specifically is required because PPO+Critic's critic-baseline advantages are small in early training (the critic predicts near the mean reward, advantages $\approx$0), so a tighter ref anchor is needed to prevent premature drift while the critic warms up.

- kl_coef = 10^{-3} for GRPO/VinePPO/CASPO (10× upstream's 10^{-4}); kl_coef = 10^{-2} for PPO+Critic (100× upstream's 10^{-4}). Both values remain inside the standard RLHF range, between InstructGPT's 0.02 and DeepSeek-Math GRPO's 0.04. Upstream VinePPO's unusually small value is paired with K=9 Monte Carlo step values that absorb V-noise we do not have.

- Standardized advantage clip $\pm 3\sigma$ retained as an outlier safety net (the upstream-faithful no-clip choice survives at VinePPO but causes CASPO to occasionally take large policy steps that destabilize online V_ϕ).

We note the 10× KL-coefficient asymmetry between PPO+Critic and the other methods as a comparability concern and report all four methods at their stable-converged KL coefficient rather than at a single shared value. At 7B, all four methods hold at the upstream kl_coef $= 10^{-4}$ without modification.

Production PRM provenance (orig recipe). The Phase-1 PRM checkpoint that drives the reported CASPO math500/gsm8k gains (the "orig" recipe in our internal log) was trained for an intended three-epoch budget of 5046 optimizer steps and the run terminated at step 4500 (≈89% of the intended budget) when a periodic save filled the output volume mid-write. The auto-save's `/best` symlink (selected on the leaky prefix-shuffle val with `best_val_loss` =0.4060) and the `/step_4500` subdir both point to the same step; we use the `/best` payload (a complete 3.55 GB `model.safetensors`), since the partial `/step_4500` write is truncated to 1.16 GB. The interrupted-budget caveat matters only for the orig PRM; subsequent refreshes complete their full-epoch budgets uneventfully under FSDP=4 with the cached reference-logprob path.

V_ϕ retrain after verifier and tokenization changes. V_ϕ's offline training data was collected before two upstream-aligned patches (Minerva-style answer-extraction cascade in the verifier; explicit BOS prepending in the rollout engine). At RL time, this leaves V_ϕ scoring *prefixes that begin with a different first token* (`<s>` vs. `[`) against *outcomes from a verifier that accepts a broader set of answer formats*. The offline-trained V_ϕ reaches step-1 `v_acc`= 0.32 against runtime, well below its offline-train accuracy of ~ 0.96. Re-running the data-collection pipeline with the live verifier and rollout engine, then retraining V_ϕ for three epochs at the same paper-faithful 5×10^{-7} offline learning rate, lifts step-1 `v_acc` to 0.74 at runtime — a 2.3× recovery from the stale-label/shifted-prompt mismatch. We use this retrained V_ϕ for all reported CASPO runs.

6 Results

This section will report wall-clock and accuracy after the full training and evaluation runs complete. The intended structure is:

- Per-method MATH-500 pass@1 and pass@16 at `final`, with seed-level error bars over a small number of seeds (target: 3 seeds per method per scale). Methods include GRPO (μ=1 canonical and μ=2 iso-budget), PPO+Critic, VinePPO, and the four CASPO configurations: CASPO-Δp, CASPO-$\Delta \log p$, CASPO-Δp+refresh, and CASPO-$\Delta \log p$+refresh.

- Per-method full-suite (MATH-500, GSM8K, OlympiadBench) pass@1 greedy and pass@16 at `final`.

- Training curves on MATH-500 sample evaluation at intermediate checkpoints (every 50 outer steps) plus `final`, summarizing sample efficiency.

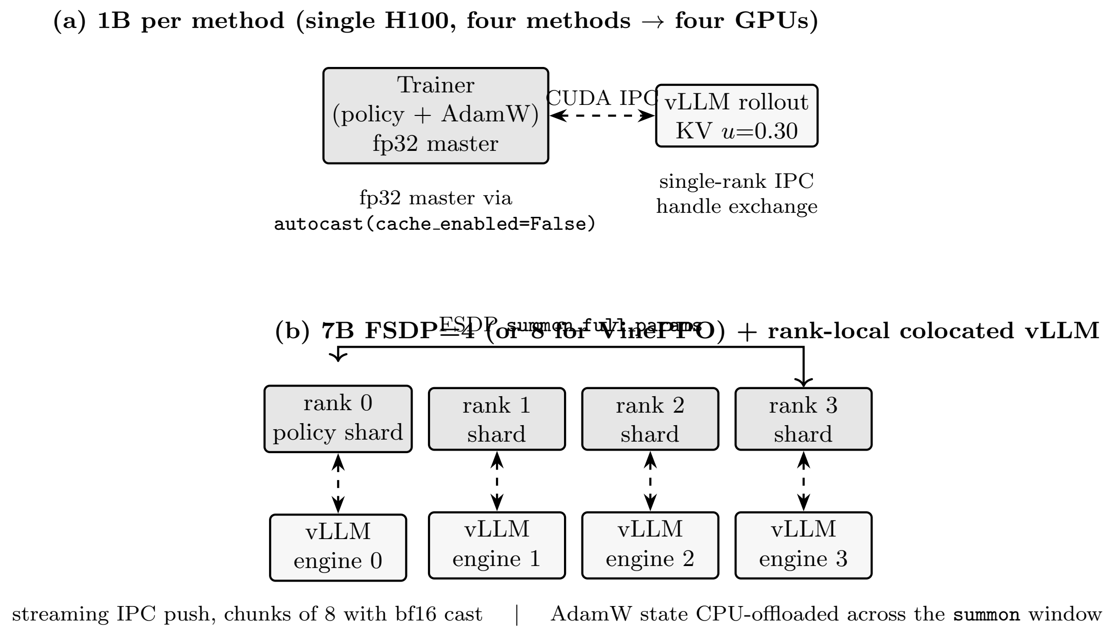

Figure 3: Trainer/rollout topology. *(a)* Each 1B run colocates trainer and vLLM on a single H100 (four methods → four GPUs of the same node); fp32 master is maintained via `autocast(cache_enabled=False)` so AdamW covers all parameters, and the IPC handle exchange between the single-rank trainer and its vLLM engine is direct. *(b)* Each 7B run uses FSDP=4 (FSDP=8 for VinePPO) with one rank-local vLLM engine per rank. Trainer-to-vLLM weight sync streams parameters in chunks of 8 with per-tensor bf16 cast inside an `FSDP.summon_full_params(writeback=False)` block; AdamW state is offloaded to pinned host memory across the summon window so the ∼28 GiB unsharded fp32 materialization fits alongside the vLLM KV cache reservation.

- Per-formulation comparison of Δp vs. $\Delta \log p$ on MATH-500 pass@16, mapping the empirical winner to the saturation regime predicted in Section 4.5 and visualized in Figure 2.
- CASPO vs. CASPO+refresh: with the no-refresh default isolating the value of the Phase-1 PRM and CASPO+refresh quantifying the additional contribution of the iterated PRM-refresh path (Section 4.4).
- PPO+Critic vs. CASPO: contrast a per-token learned scalar baseline against a per-step learned prefix value, both under the same PPO objective and the same global PPO minibatch.
- Wall-clock summary: per-method step time, per-method 1000-step ETA, and the implied GPU-hour cost of the four-method comparison, with the breakdown by phase (rollout, old-logprob rescore, value/critic forward, ref-logprob precompute, policy update, IPC sync).

We report all numbers under the same training contract: identical SFT initialization, identical prompt set, identical verifier, identical group/response budget, identical PPO clipped objective, identical KL coefficient, identical advantage standardization, identical max sequence length, and identical evaluation sampling parameters. The only intended difference between rows is the source of the per-step value $V(s_t)$ in (1): zero with a per-prompt group baseline (GRPO), a learned per-token scalar value $V_\psi(s,t)$ (PPO+Critic), Monte Carlo continuations (VinePPO), or a learned prefix value $V_\phi(s_t)$ (CASPO).

7 Analysis

This section will examine, given the empirical results, what determines when V_ϕ matches VinePPO's prefix-rollout signal and when it does not. The intended structure is:

- *Step-credit calibration*: do step advantages from V_ϕ rank correct vs. incorrect step continuations consistently with on-policy MC?
- *PRM decay vs. refresh*: how quickly does the Phase-1 V_ϕ lose calibration as the policy drifts (Spearman ρ vs. fresh MC labels per outer step), and how much of that decay does the CASPO+refresh schedule recover at each refresh trigger?
- *Wall-clock per accuracy point*: the practical question CASPO is asked to answer is whether learned prefix values can match VinePPO's accuracy at a fraction of the wall-clock cost. We report this as accuracy-versus-GPU-hours for each method.
- *Step-count effects on VinePPO*: VinePPO's wall-clock scales as $K \cdot$ steps/response, so the marginal cost of step-level credit grows with response length. We quantify this by binning the test set into long vs. short reasoning regimes and reporting the per-bin accuracy/wall-clock trade-off.

8 Limitations

Reward-model drift. A V_ϕ trained on the SFT distribution decays in calibration as the policy optimizes; the prefixes encountered later in training are no longer drawn from the SFT marginal, and Spearman ρ vs. fresh MC labels declines non-monotonically with a sharp mid-RL drop (Section 4.4). CASPO+refresh addresses this by periodically retraining V_ϕ from scratch on Monte-Carlo-labeled rollouts of the current policy (every k=150 outer steps for Δp, k=200 for $\Delta \log p$). The amortized refresh overhead is on the order of 10–20 s per RL outer step (Section 4.4), well below one VinePPO K=9 step's $\sim$33 min, so the method remains dominated by RL rollout cost rather than refresh overhead. Neither the no-refresh default nor refresh is a complete defense against pathological feedback loops (e.g. a value head that learns to predict policy surface form rather than correctness); we report both to quantify the contribution of the refresh path.

Cross-distribution probe-set evaluation of V_ϕ. Selecting the best refresh recipe by ranking candidate V_ϕ checkpoints on a fixed probe set is sensitive to the response-length cap used when the probe data was generated. A V_ϕ trained on collection budget L_{tr} sees prefixes spanning the range $[0, L_{\text{tr}}]$, but a probe set generated at cap $L_{\text{probe}} < L_{\text{tr}}$ contains prefixes only in $[0, L_{\text{probe}}]$. Differences in Spearman ρ between checkpoints trained at different L_{tr} on a single shared probe set therefore conflate intrinsic credit-assignment quality with the fraction of the checkpoint's training distribution that the probe never exercises. We encountered this confound in early refresh-recipe sweeps and recommend that any future V_ϕ recipe selection generate the probe at the maximum training cap of any candidate, or otherwise restrict the comparison to recipes whose training caps match the probe cap.

Segmenter quality. The LaTeX-aware step splitter is a structural heuristic. Manual spot checks on Qwen2.5-Math-1.5B and DeepSeekMath-7B generations show coherent boundaries for prose, equations, factorization, and aligned derivations, but generations that continue past a final boxed answer have those trailing fragments treated as additional steps; a post-answer truncation pass is left as future work.

Apples-to-apples scope. Our comparison fixes the SFT initialization, prompt set, verifier, sampling parameters, KL constraint, and advantage standardization. It does not exhaustively explore method-specific tuning (e.g. different K for VinePPO, different β/m for CASPO, larger G for GRPO); we report the upstream-paper values where available and otherwise the values that minimize wall-clock without regressing pass@1 on a small validation set.

Scale. Results at 1.5B and 7B may not transfer to substantially larger scales. The 7B FSDP+vLLM-IPC stack is novel to this work; whether the same configuration extrapolates to 70B is an open systems question.

Single-seed PRM ranking. Our PRM A/B comparisons (Phase-1 SFT-source vs. policy-source refresh, including the per-stage specialization in Section 4.4) are based on one trained PRM per recipe. A paired-t test across the eight CASPO RL checkpoints we log shows that none of the recipe-level metric-B ρ differences are significant at $p < 0.05$; the 95% confidence intervals all straddle zero, even though per-checkpoint deltas are large in magnitude ($\approx$0.10–0.20 ρ). Per-stage specialization is real but cancels in the trajectory average. Multi-seed PRM training over ≥ 3 independent label collections is the only honest path to a definitive recipe ranking; the comparisons reported here are suggestive, not proven, and any "wins by X ρ" phrasing should be read with that caveat.

9 Future Work: Optimizing the Success-Prediction Architecture

Our results adopt IPVRM's cumulative-log-ratio parameterization for V_ϕ unchanged, after empirically confirming it dominates a single-sigmoid-head alternative under matched hyperparameters (Section 2). The architectural margin appears to come from gradient density — the log-ratio decomposes loss as a sum of per-token contributions, so every response token in the prefix supplies a direct gradient signal — but several design questions remain open within this regime.

Heads, depths, and ensembles. We trained V_ϕ with a single learnable backbone whose log-ratio is read directly. Future work should compare (i) deeper non-linear value heads on top of the cumulative log-ratio (e.g. a 2–3 layer MLP that maps $V_\phi(s_t)$ to a scalar logit), (ii) Q-style critics that score each per-token continuation rather than a prefix value, and (iii) ensembles of V_ϕ trained with different seeds or label collections, both as variance-reduction tools and as honest-uncertainty signals for the step-TD advantage.

Differential learning-rate schedules. Both architectures we report use a single LR over the entire backbone. Standard practice for fresh value heads is a 10–100$\times$ higher LR than the pretrained backbone with a brief warmup; this could close some of the gap on the sigmoid-head baseline and may also help the log-ratio PRM at smaller training-data sizes. We did not sweep this axis.

Training-data form. We use Math-Shepherd-style continuous-target BCE on per-prefix Monte Carlo $\widehat{p}$ with $J = 8$ continuations and $S = 5$ step prefixes per response. Promising variants include: increasing J for tighter $\widehat{p}$ variance at fixed compute through bucketed batching; calibration-aware losses (focal BCE, temperature-scaled targets) when the underlying success-rate distribution is bimodal; and contrastive losses pairing within-prompt prefixes of different success rates.

Continuation count J at fixed checkpoint-selection. The orig recipe uses J=16 continuations per prefix; refreshes use J=8 to halve label-collection cost. A direct A/B at fixed recipe (orig at J=8 vs. orig at J=16) shows that on the *leaky-val-selected* `/best` checkpoint J=16 leads J=8 by $\approx$0.027 ρ on average metric-B; but at the *metric-B-selected* peak (which falls at $\approx$60% of the optimizer budget, well before the leaky-val-selected `/best`), J=8 ties J=16 on metric-B (paired-t p=0.93). The apparent J=16 advantage is therefore an artifact of leaky-val mis-selection rather than a structural property of the larger continuation count: when the selector is non-leaky (or when training stops at the metric-B peak), J=8 is sufficient. This is a useful corollary of the earlier statistical-significance caveat and a concrete recipe simplification (halve J at no measurable metric-B cost).

Inference-time refinements. At inference, V_ϕ is queried in a single forward pass. Test-time ensembling (multiple seeds, MC-dropout, temperature scaling on V_ϕ/β) and online calibration on a small held-out probe set during RL could improve both the calibration and the action of the step-TD advantage on CASPO performance.

Held-out probe protocol. The empirical comparison reported here uses a prompt-level held-out split (`--split_by_prompt` or `--held_out_data`); the prefix-level row shuffle that the trainer still supports as a legacy default exhibits same-prompt leakage and should not be used for any load-bearing claims about V_ϕ generalization. Future PRM ablations should default to held-out prompts and quantify the gap between in-distribution and OOD performance as a single reported column.

Minimum RL training-set size for benchmark transfer. We train RL on the full 1209-prompt `dsr_sub` set and observe strong generalization to MATH-500, GSM8K, OlympiadBench, AMC, AIME, and `college_math` despite the training-prompt count being orders of magnitude smaller than typical instruction-tuning corpora.

Asymmetric generalization requirement. A subtle point that this result foregrounds: the PRM and the policy face very different generalization demands. The PRM is only ever queried on prefixes the policy generates during RL, and those prefixes always come from the same fixed RL prompt set; the PRM never has to extrapolate to a new prompt distribution. The policy, by contrast, must transfer from those same RL prompts to the actual evaluation benchmarks. This is exactly the asymmetry our held-out comparison exposes: the same V_ϕ that held-out OOD evaluation flags as having weak rank-correlation on unseen prompts is nonetheless the same V_ϕ that drives useful step-TD advantages *on its own training prompts* during RL, where it does not need to generalize. Crucially, the policy's benchmark transfer is then carried by the underlying base SFT representations and by the structural reasoning patterns RL reinforces, not by the PRM extrapolating to new problems. So an in-distribution-only PRM trained on a small high-quality RL set is not a weakness — it is the operating regime the design tolerates.

Practical question. How small can the RL prompt set be while still yielding the same benchmark transfer? A 300-prompt RL set, for instance, would cut both the PRM training wall-clock and the RL rollout wall-clock by roughly 4× relative to the 1209-prompt default — both phases scale roughly linearly with prompt count at fixed K, J, and rollouts per RL step — and the iterated-refresh schedule (§4.4) would amortize even further if the per-iteration PRM is cheaper. The asymmetric-generalization framing above makes this plausible: a tight PRM on a small RL set is enough because the PRM only has to be calibrated *on that set*; the policy generalizes via the base LM features.

Promising next experiments. (i) Sweep prompt-set sizes $N \in \{100, 300, 600, 1209\}$ at fixed RL budget and report the benchmark-transfer Pareto curve, distinguishing PRM held-out ρ (which is allowed to be low) from policy benchmark pass@1 (which must remain high). (ii) Compare a fixed random subset to a curated difficulty-balanced subset at the same N (since success-rate distributions shift with the prompt mix, per Section 4.1). (iii) Decouple the budgets entirely: a *small* high-quality RL prompt set paired with a *larger*-collection PRM. The structural advantage of CASPO over methods that require on-policy MC continuations (VinePPO) is exactly that the PRM is decoupled from the RL prompt set, so the two budgets can be chosen independently.

10 Conclusion

We described CASPO, a step-level credit assignment method for reasoning RL that replaces VinePPO's on-policy Monte Carlo prefix rollouts with a learned process reward model trained on Monte Carlo step labels to predict $P(\text{success} \mid \text{prefix})$, and a periodic refresh of that model on current-policy rollouts to control distribution drift. CASPO fits cleanly into the same PPO clipped objective, the same KL constraint, and the same step-TD advantage construction used by VinePPO, isolating the credit-signal change from confounders. By unifying GRPO, PPO+Critic, VinePPO, and CASPO under one trainer, one verifier, and one rollout engine, we make it possible to compare these four methods under identical conditions on Qwen2.5-Math-1.5B with `dsr_sub` as the primary site and DeepSeekMath-7B with MATH-lighteval as a paper-faithful 7B reference. Empirical results in Section 6 will quantify whether the learned value model recovers VinePPO's reasoning-credit signal at a fraction of its generation cost, or whether step-level Monte Carlo remains uniquely advantageous, and whether iterated refresh of V_ϕ from current-policy rollouts closes any residual gap.

References

[1] Shiping Gao, Hongzhan Chen, Xiaojun Quan, Qifan Wang, and Lifu Huang. Unleashing implicit rewards: Prefix-value learning for distribution-level optimization. *arXiv preprint arXiv:2604.13197*, 2026. URL `https://arxiv.org/abs/2604.13197`.

[2] Amirhossein Kazemnejad, Milad Aghajohari, Eva Portelance, Alessandro Sordoni, Siva Reddy, Aaron Courville, and Nicolas Le Roux. Vineppo: Unlocking rl potential for llm reasoning through refined credit assignment. *arXiv preprint arXiv:2410.01679*, 2024. URL `https://arxiv.org/abs/2410.01679`.

[3] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS Symposium on Operating Systems Principles*, 2023. URL `https://arxiv.org/abs/2309.06180`.

[4] McGill NLP. Vineppo codebase. `https://github.com/McGill-NLP/VinePPO`, 2024. Code for VinePPO: Unlocking RL Potential For LLM Reasoning Through Refined Credit Assignment.

[5] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. URL https://arxiv.org/abs/1707.06347.

[6] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, Daya Guo, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. URL https://arxiv.org/abs/2402.03300.